

Igra nasprotnika: Pokrivanje košev

Opis algoritma

Dejan Jarc, Jure Zupančič

7. april 2026

1 Uvod

V nalogi rešujemo problem pokrivanja košev v eni dimenziji. Na vhodu dobimo zaporedje predmetov, kjer ima vsak predmet podano velikost z realnim številom med 0 in 1. Cilj je predmete razporediti v koše tako, da je vsota velikosti predmetov v čim več koših vsaj 1. Tak koš štejemo kot pokrit.

Naš problem se od klasičnega pokrivanja košev razlikuje v tem, da mora algoritem predmete razvrščati sproti ob branju z vhoda. To pomeni, da predmete obravnava v vrstnem redu, kot so podani na vhodu in mora za vsak predmet takoj določiti, v kateri koš ga bo postavil. Predmetov zato ni dovoljeno urejati po velikosti in naknadno razvrščati po koših.

2 Osnovna ideja algoritma

Za implementacijo algoritma smo uporabili požrešni pristop. Za vsak nov predmet algoritem pregleda trenutno odprte koše in poskuša izbrati najbolj primeren koš glede na trenutno stanje.

Osnovna ideja algoritma je naslednja:

1. Preberemo predmet na vhodu.
2. Če lahko trenutni predmet z dodajanjem v nek odprt koš ta koš pokrije, algoritem izbere najboljši tak koš.
3. Če trenutni predmet ne pokrije nobenega odprtega koša, algoritem izbere koš, ki ga je smiselno dodatno napolniti.
4. Če primeren koš ne obstaja, algoritem odpre nov koš in vanj postavi trenutni predmet.

Ko je koš enkrat pokrit, ga algoritem zapre in vanj ne dodaja več novih predmetov.

3 Struktura programa

Program sestavljata dva glavna dela:

- razred `bin_cover`, ki skrbi za branje vhodnih datotek, klic reševalnika in zapis izhodnih datotek.
- razred `BinCoveringSolver`, ki vsebuje glavni algoritem za polnjenje košev.
- razred `Bin`, ki predstavlja posamezen koš. Ta hrani:
 - `itemIndices`: indeksi predmetov, ki so bili dodeljeni temu košu,
 - `total`: trenutno vsoto velikosti predmetov v košu,
 - `covered`: informacijo, ali je koš že pokrit.

4 Potek algoritma

Za vsak vhodni predmet algoritem izvede naslednje korake:

1. pregleda trenutno odprte koše,
2. preveri, ali je postavitve predmeta v posamezen koš dovoljena,
3. če dodajanje predmeta v nek koš povzroči, da vsota v košu zadostuje pogoju pokritosti, je ta koš kandidat za pokritje,
4. če tak kandidat obstaja, algoritem izbere najboljšega,
5. če kandidat za pokritje ne obstaja, algoritem izbere najboljši še nepokrit koš za nadaljnje polnjenje,
6. če noben koš ni primeren, odpre nov koš.

Na koncu program v izhodno datoteko zapiše samo pokrite koše, ki jih predstavimo tako, da naštejemo indekse vsebovanih predmetov.

5 Hevristika

Ker uporabljamo požrešni pristop, ni možno preiskati celoten prostor možnih pokritij in zato ne moremo zagotoviti optimalne rešitve. Program zato uporablja hevristiko za izbiro najbolj primerne rešitve.

Pri izbiri koša upoštevamo naslednje ideje:

- če je mogoče koš pokriti, se temu običajno da prednost,
- zelo velik predmet se ne dodaja v premalo napolnjen koš,
- pri dodajanju velikega predmeta se omeji preveliko preseganje meje napolnjenosti.

S takim pogledom na pokrivanje smo zmanjšali možnosti, da bi velik predmet porabili za koš, ki bi ga bilo mogoče kasneje dopolniti z manjšim predmetom.

6 Parametri algoritma

Algoritem uporablja več parametrov, s katerimi določimo obnašanje hevrstike:

- `largeItemThreshold`: meja, nad katero predmet štejemo za velik,
- `largeReuseMinLoad`: minimalna napolnjenost koša, da smemo vanj dodati velik predmet,
- `largeItemMaxOvershoot`: največje dovoljeno preseganje meje 1 pri dodajanju velikega predmeta,
- `largeItemMinBinLoad`: meja, pod katero koš velja za preveč prazen za dodajanje velikega predmeta.

S temi parametri lahko prilagodimo delovanje algoritma glede na tip vhodnih primerov.

7 Časovna zahtevnost

Pri vsakem predmetu algoritem pregleda vse trenutno odprte koše. Če je število predmetov n in odprtih košev $k : k \leq n$, je časovna zahtevnost $O(n \cdot k)$, saj se lahko pri vsakem predmetu pregleduje večje število odprtih košev. V najslabšem primeru, ko je $k = n$, je časovna zahtevnost $O(n^2)$.

Prostorska zahtevnost je $O(n)$, ker hranimo podatke dodeljenih predmetih po pokritih koših.

8 Zaključek

Naša rešitev za pokrivanje košev temelji na preprostem požrešnem algoritmu, ki je primeren za nalogo, kjer predmetov ni dovoljeno preurejati. Glavna prednost pristopa je enostavna implementacija ter uporaba hevrstike za velike predmete, ki izboljša kakovost rezultatov v primerjavi z naivnim polnjenjem košev.